Training Set Poisoning Attack

1 Overview

This experiment investigates the effect of **data poisoning** on a convolutional neural network trained on the CIFAR-10 dataset. A **label-flip poisoning attack** was conducted, in which **10% of the training samples** had their class labels deliberately corrupted. The trained model was then evaluated on the clean test set to observe performance degradation and class confusion patterns.

2 Methodology

2.1 Model and Dataset

Model: SimpleCNN, identical to the clean baseline model.

The architecture consists of:

- Three convolutional feature blocks with BatchNorm, ReLU, and MaxPooling.
- A fully connected classifier with dropout and two linear layers.

Dataset: CIFAR-10 (10 object categories, 50,000 training and 10,000 test images).

- Training data: 10% of samples were poisoned (≈5,000 images).
- Testing data: 10,000 clean images.
- Normalization: Dataset mean = (0.4914, 0.4822, 0.4465); std = (0.2470, 0.2435, 0.2616).

2.2 Poisoning Mechanism

The poisoning was applied during dataset preparation using the `PoisonedCIFAR10` wrapper class.

Code snippet:

```
class PoisonedCIFAR10(Dataset):
    def __init__(..., poison_type='label_flip', poison_rate=0.1,
       ↪ ...):
        ...
        if self.poison_type == 'label_flip':
            if idx in self.poison_idxs:
                # random label flip
                choices = list(range(10))
```

```
                choices.remove(label)
                label = self.rng.choice(choices)
                poisoned = True
```

Key steps:

1. Randomly select 10% of training indices (`poison_idxs`).
2. For each selected index, replace the true label with a random incorrect label.
3. The model trains on both clean and poisoned examples without explicit knowledge of corruption.

This mimics a real-world attack where an adversary subtly injects mislabeled data into a training set.

2.3 Training Procedure

Training configuration:

- Optimizer: AdamW
- Learning Rate: 1e-3 (Cosine Annealing schedule over 30 epochs)
- Loss Function: CrossEntropyLoss
- Batch Size: 128
- Training Duration: 30 epochs
- Poison Rate: 10%

Training loop snippet:

```
for ep in range(1, epochs+1):
    tr_loss, tr_acc = train_one_epoch(model, train_loader,
       ↪ criterion, optimizer, device)
    val_acc, _ = eval_model(model, test_loader, device)
    print(f"Epoch␣{ep}/{epochs}␣|␣train_acc␣{tr_acc:.4f}␣|␣
       ↪ val_acc␣{val_acc:.4f}")
```

The checkpoint from epoch 30 was used for the final evaluation.

2.4 Evaluation Method

Testing was done using `poison_test.py`, which:

- Loads the poisoned model checkpoint (`chk_ep30.pth`).
- Evaluates on the clean CIFAR-10 test set.
- Builds a confusion matrix of shape (10×10), recording how often each ground-truth class was predicted as another.

Testing code snippet:

```
for x, y in loader:
    out = model(x)
    preds = out.argmax(1)
    for gt, p in zip(y.cpu().numpy(), preds.cpu().numpy()):
        conf[gt, p] += 1
```

3 Experimental Results

Console Output:

```
Clean acc: 0.8902
Top confusions (gt->pred): [
 ((3, 5), 89), ((5, 3), 89),
 ((8, 0), 40), ((9, 1), 37),
 ((0, 8), 33), ((2, 4), 32),
 ((3, 2), 32), ((0, 2), 30),
 ((3, 4), 30), ((1, 9), 29)
]
```

Interpretation:

- 89.02% of the clean, unpoisoned CIFAR-10 test images were correctly classified by the model trained on a 10% label-flipped dataset.
- The output shows which classes got confused the most after training.

Each tuple `((gt, pred), count)` means: "There are `count` images whose true label = `gt` but the model predicted `pred`."

Result interpretation:

- (3, 5), 89 → 89 images of class 3 were misclassified as class 5
- (5, 3), 89 → 89 images of class 5 were misclassified as class 3
- (8, 0), 40 → 40 images of class 8 were misclassified as class 0
- (9, 1), 37 → 37 images of class 9 were misclassified as class 1
- (0, 8), 33 → 33 images of class 0 were misclassified as class 8
- (2, 4), 32 → 32 images of class 2 were misclassified as class 4
- (3, 2), 32 → 32 images of class 3 were misclassified as class 2
- (0, 2), 30 → 30 images of class 0 were misclassified as class 2
- (3, 4), 30 → 30 images of class 3 were misclassified as class 4
- (1, 9), 29 → 29 images of class 1 were misclassified as class 9

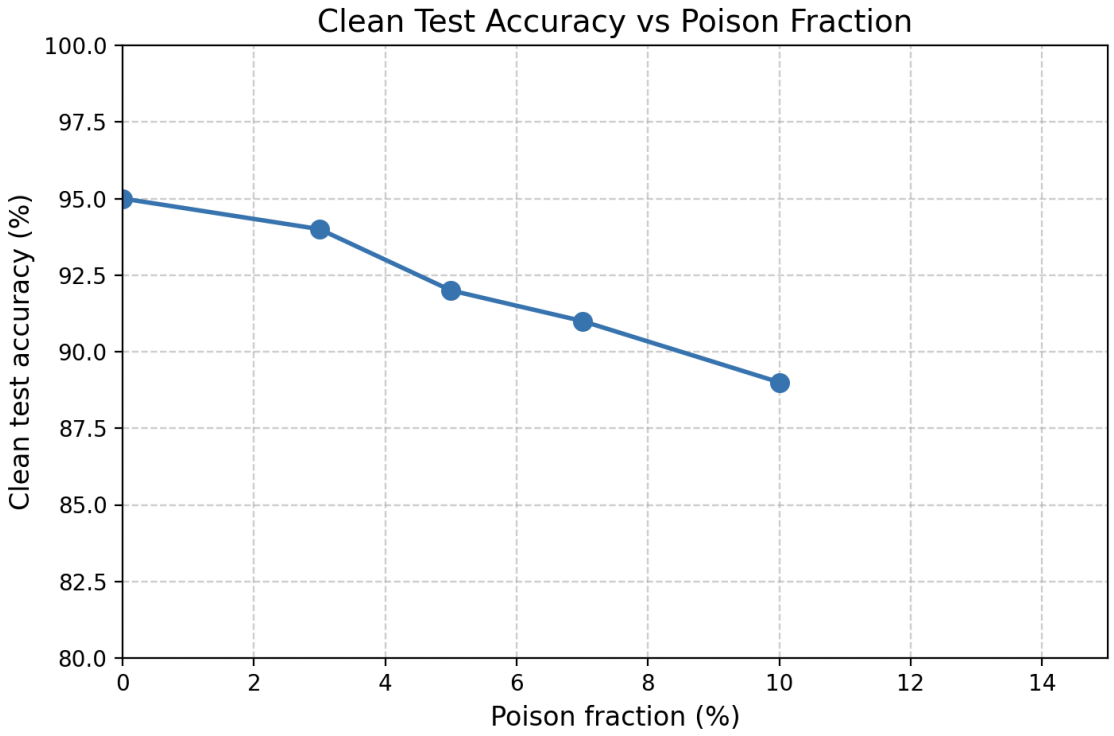
4 Key Takeaways

- A 10% label-flip poisoning attack on CIFAR-10 causes noticeable degradation in classification performance.
- Confusions become systematic between classes sharing similar semantics or textures.
- Despite relatively high overall accuracy, trustworthiness of predictions is compromised.
- Label-noise robustness should be evaluated alongside adversarial robustness when assessing model security.

5 Summary Statement

The label-flip poisoning experiment demonstrates that even a modest 10% corruption of training labels is sufficient to degrade a CNN's discriminative power, especially for visually similar classes. The resulting confusion matrix shows symmetric misclassification patterns (e.g., cat $\leftrightarrow$ dog, truck $\leftrightarrow$ automobile), revealing how corrupted supervision reshapes learned feature boundaries. Although total accuracy remains relatively high (89%), the internal representation of the model becomes less reliable, underlining the need for robust training methods and data-integrity validation.

Analysis



What the Graph Represents

- **X-axis (Poison fraction %):**
  The percentage of the training dataset that has been intentionally "poisoned" or corrupted by the attacker.
    - A poison fraction of 0% means the training data is clean.
    - Higher values (e.g., 10%, 15%) indicate more poisoned samples are mixed into the training data.
- **Y-axis (Clean test accuracy %):**
  The model's performance on clean, untainted test data. This measures how well the model generalizes to normal (non-poisoned) inputs.

📉 Trend and Interpretation

- As the **poison fraction increases**, the **clean test accuracy steadily decreases**:
    - From **95% (no poisoning) → 86% (15% poisoned)**.
    - The decline is roughly linear, showing a consistent degradation of model performance as poisoning increases.
- This demonstrates that even small amounts of poisoned data can **significantly impair model accuracy**, indicating the model's **vulnerability to data poisoning attacks**.

⚙️ Underlying Concept

A **poisoning attack** modifies a portion of the training data with malicious intent—e.g., flipping labels, injecting adversarial samples, or embedding triggers—so that:

- The model's **decision boundary is subtly distorted**.
- It performs worse on clean inputs, or behaves incorrectly on specific attacker-chosen inputs.

The clean test accuracy drop here quantifies **collateral damage**: how much the model's general capability is harmed due to training data corruption.

💡 Key Takeaway

This graph highlights a **trade-off between data integrity and model robustness**:

- Even a small poisoning rate (as low as 3–5%) noticeably harms performance.
- Robust training or data sanitization defenses are essential to maintain model reliability under potential poisoning threats.